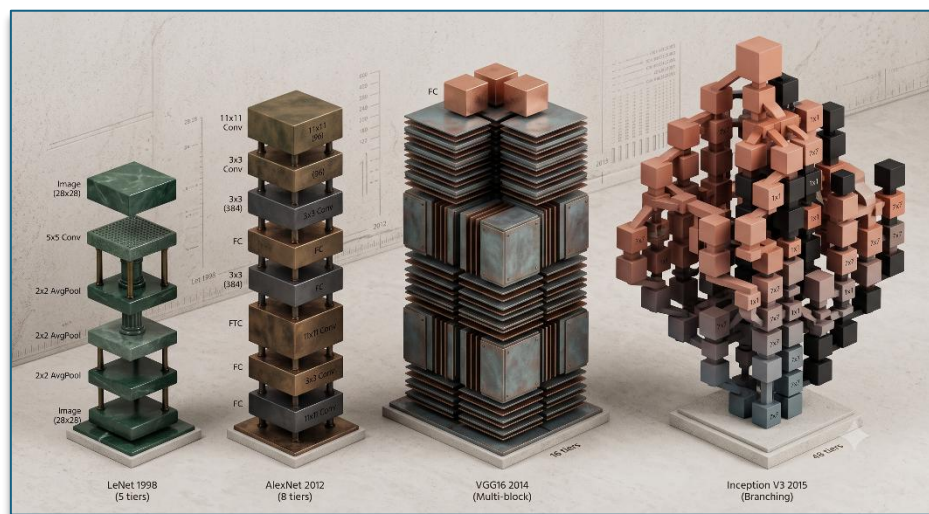


Batch-Layer Norms and Residual Connections

Overview

- implement batch normalization to accelerate training
- utilize residual connections to build deeper networks

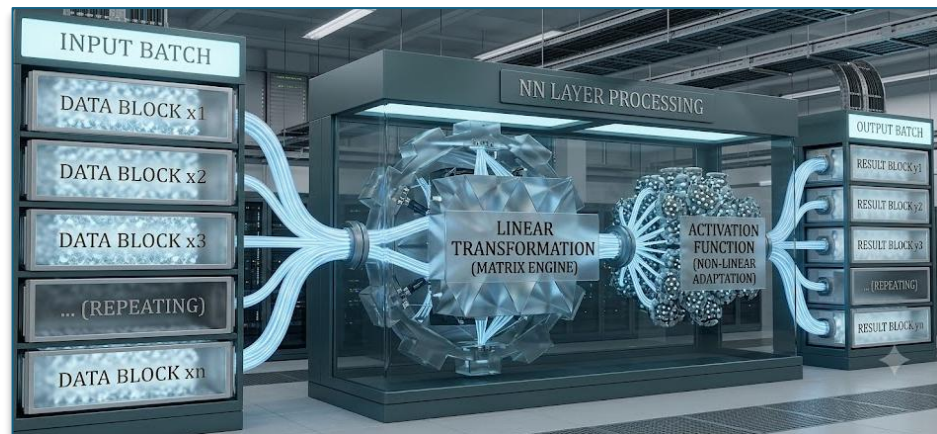
Async

Training Really
Deep NNs

Gemini 3.5

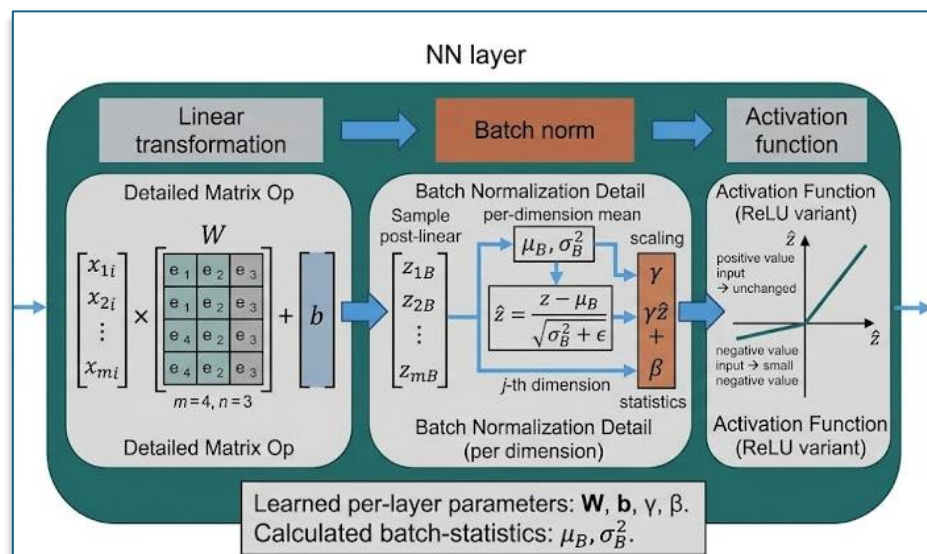
- NNs have been increasing in complexity and numbers of layers over time

Batch Normalization



Gemini 3.5

- the input batch is similar to a mini batch with stochastic gradient descent, essentially it is a set of examples along which we want to make an update in gradient descent
- the output batch becomes the input batch for the next layer



Gemini 3.5

- with batch normalization, once the data has been transformed linearly, a standardization operation is performed before being passed to the non-linear activation function
- alternatively, batch normalization can be after NL activation

- to compute batch normalization:

- compute the batch mean:

$$\hat{\mu}_{\mathcal{B}} = \frac{1}{|\mathcal{B}|} \sum_{x \in \mathcal{B}} x$$

computes
average vector
input

- compute the batch variance:

$$\hat{\sigma}_{\mathcal{B}}^2 = \frac{1}{|\mathcal{B}|} \sum_{x \in \mathcal{B}} (x - \hat{\mu}_{\mathcal{B}})^2$$

computes
variance of
vectors

- normalize each batch sample:

$$BN(x) = \frac{x - \hat{\mu}_{\mathcal{B}}}{\hat{\sigma}_{\mathcal{B}}}$$

normalization
with mean = 0
and s.d. = 1

- it is unclear if normalizing to mean = 0 and s.d. = 1 and can lead to division by zero in the third equation if the batch variance is zero
- to avoid this, we add a small amount of noise (ϵ) to the second equation

$$\hat{\sigma}_{\mathcal{B}}^2 = \frac{1}{|\mathcal{B}|} \sum_{x \in \mathcal{B}} (x - \hat{\mu}_{\mathcal{B}})^2 + \epsilon$$

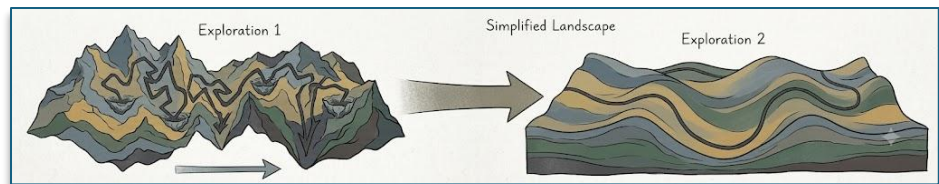
- additionally, we add two parameters (vectors γ and β) to the third equation

$$BN(x) = \gamma \odot \frac{x - \hat{\mu}_{\mathcal{B}}}{\hat{\sigma}_{\mathcal{B}}} + \beta$$

- we do an element-wise product of each batch normalized example with γ to re-scale and constant vector β which shifts the

value of the normalized batch and redefines to a different mean and variance

- during test time, this is done somewhat differently by using the values collected during training to estimate reasonable values for μ and σ
- batch normalization for images
 - for each channel μ and σ are calculated and normalized separately
- benefits of batch normalization
 - smooths the optimization landscape
 - can use a larger learning rate – faster convergence



Gemini 3.5

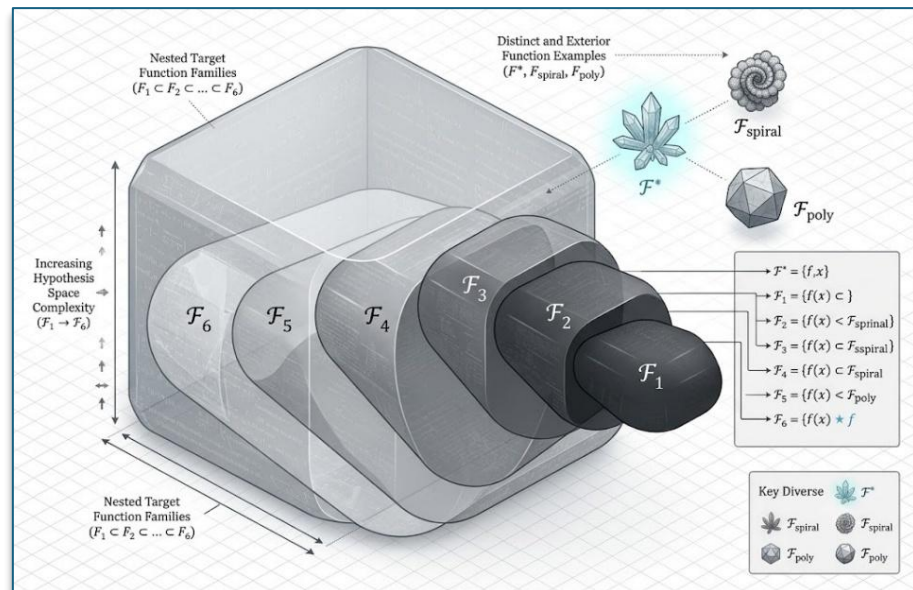
- regularization from noisy estimates of mean and variance
 - “sweet spot” for batch size
 - batch too small
 - noisy mean and variance estimates
 - batch too large
 - mean and variance estimates not noisy enough

Layer Normalization

- with layer normalization, we don't compute aggregated statistics across a batch, each example in the batch is normalized independently
- benefits of layer normalization
 - inputs to layer never explode or vanish
 - no dependence on batch size
 - forward pass during training and testing are the same

Residual Connections

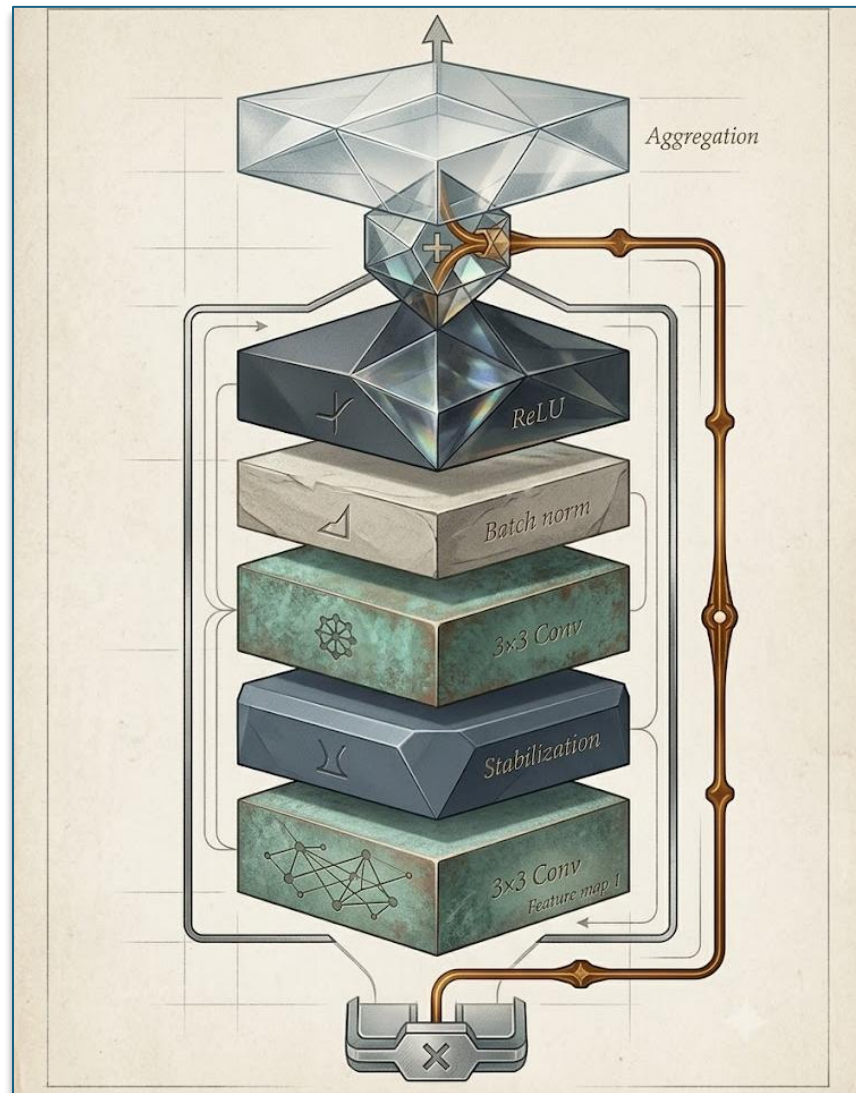
- let F be the set of functions an NN architecture can represent
- let f^* be the function we want to learn



Gemini 3.5

- each layer is a set of functions that can be represented by our feed forward network and outputs to the next layer with each layer's functions increasing in complexity
- this leads to a situation where more complex layers become disjointed from the simpler architectures

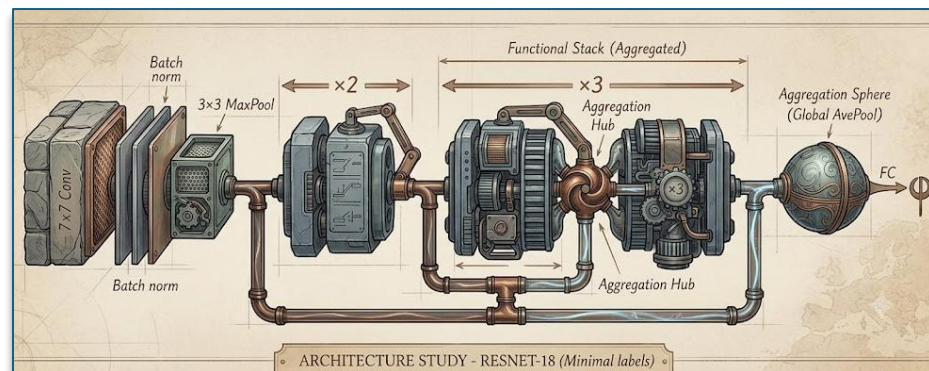
- ideally, the more complex architectures should already encapsulate the simpler architectures absorbing the horizons of the lower layers
- to guarantee that the $n + 1$ layer NN can represent a superset of the functions that the n layer NN can represent, make sure that the $n + 1$ layer can represent the identity function



Gemini 3.5

- several convolution layers, activation, and batch normalizations are used and at the end, you add a residual connection that bypasses this computation and adds x back into loop

- this implies that with the residual connection, we can always implement the identity function ensuring that computations made by a smaller network can also be performed by any larger network
- additionally, a 1×1 convolution can be added to the residual loop before it is passed



Gemini 3.5

- many modern architectures for deep learning and specifically for computer vision have moved towards uniform computation layer blocks that repeat with ResNet connections across each block
- benefits of residual connections
 - expands the set of functions that a NN can represent
 - prevents “shattering gradients”



Gemini 3.5

- in a shallow network, points close together on the x-axis have similar gradients
- extending to deeper architectures, the pattern breaks and there is very little correlation between adjacent points resulting in a “spiky” network

	▪ by adding residual connections, this spiky-ness is normalized producing a better fitting model
Summary	• increasing depth has proven to be a reliable way of improving NNS • bath normalization can layer normalization smooth the optimization landscape by normalizing activation/pre-activation • residual connections smooth the optimization landscape and ensure that adding layers expands the set of representable functions • normalization and residual connections are important components of sequence-modeling architectures such as RNNs transformers